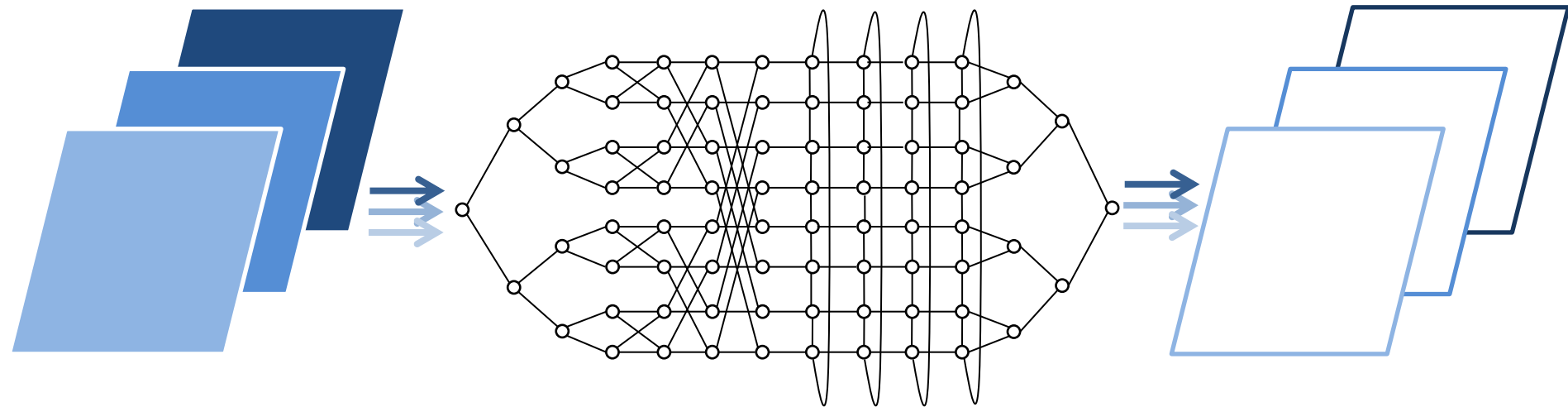


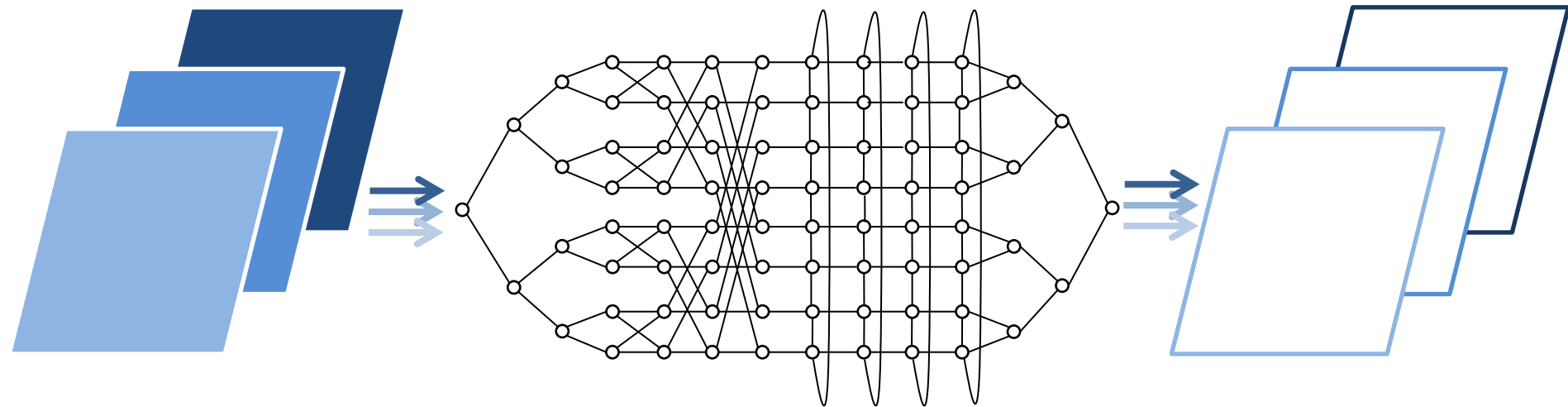
Programmation Parallèle

MPI: Message Passing Interface

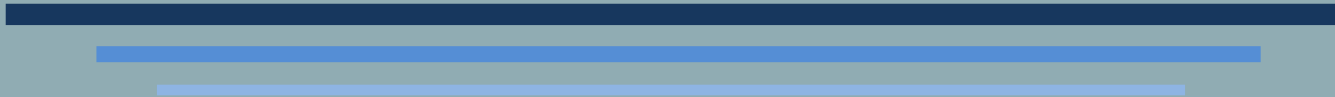


SCALABILITY and Advanced communications

ENSIIE-HPC/BigData-PP-IIP-Lecture 3



SCALABILITY ISSUES



MPI: Message Passing Interface

Optimizing a parallel program



- Optimizing a parallel program is more challenging
- The goal is to be the closest possible to a “perfect” parallel program
- A perfect parallel program means if you double the number of workers, your program will run twice as fast
- An everyday example – sorting things:

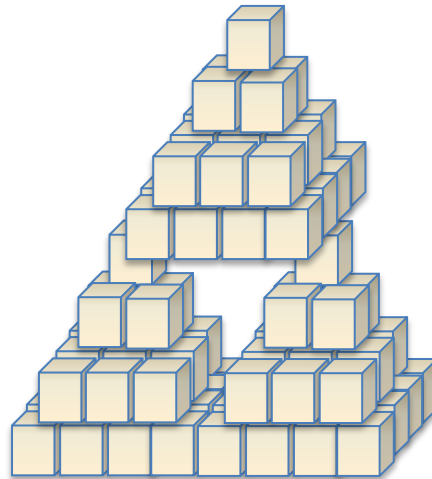
Optimizing a parallel program



- Sorting things:

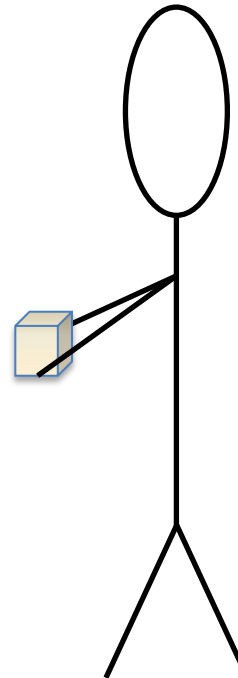
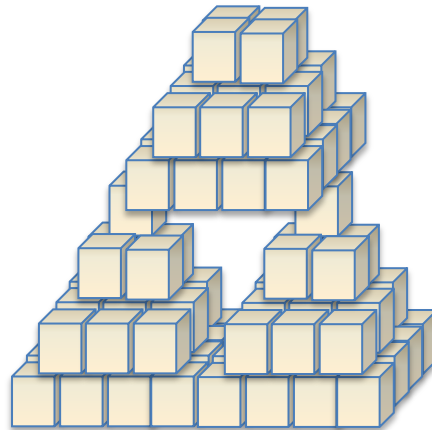
Optimizing a parallel program

- Sorting things:
 - Consider a pile of 90 boxes to sort



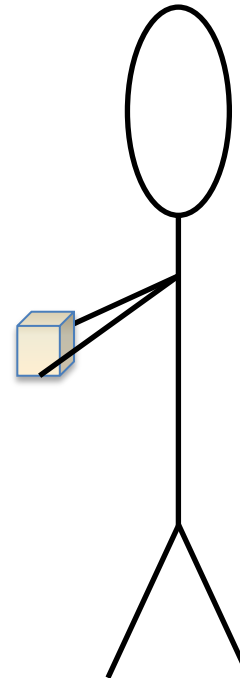
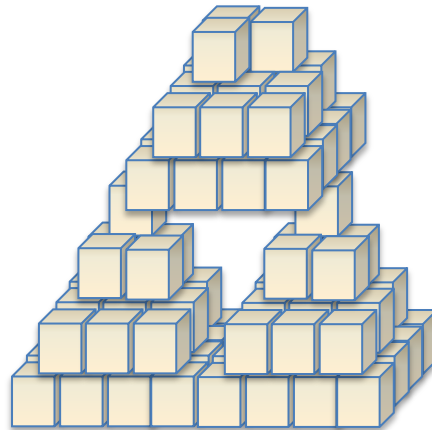
Optimizing a parallel program

- Sorting things:
 - Consider a pile of 90 boxes to sort
 - If it takes 10 seconds for one person to handle a box



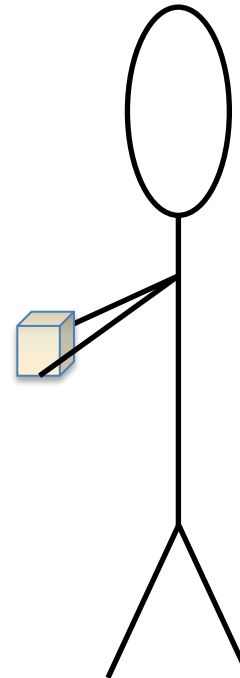
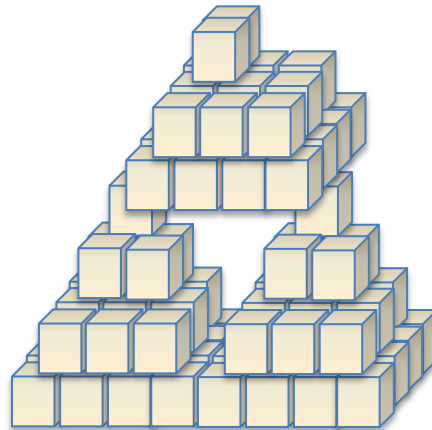
Optimizing a parallel program

- Sorting things:
 - Consider a pile of 90 boxes to sort
 - If it takes 10 seconds for one person to handle a box
 - it



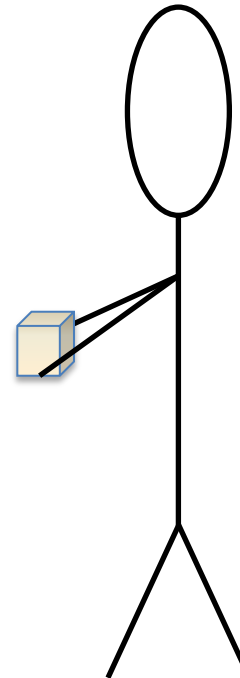
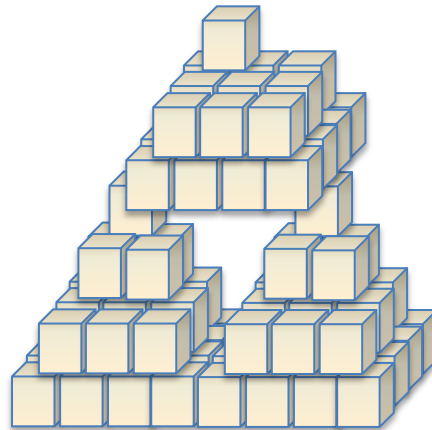
Optimizing a parallel program

- Sorting things:
 - Consider a pile of 90 boxes to sort
 - If it takes 10 seconds for one person to handle a box
 - It will



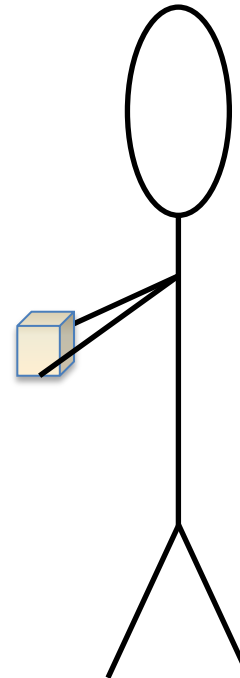
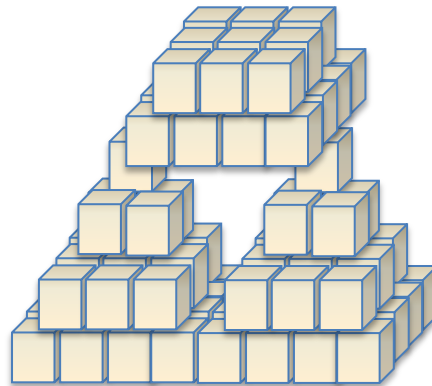
Optimizing a parallel program

- Sorting things:
 - Consider a pile of 90 boxes to sort
 - If it takes 10 seconds for one person to handle a box
 - It will take



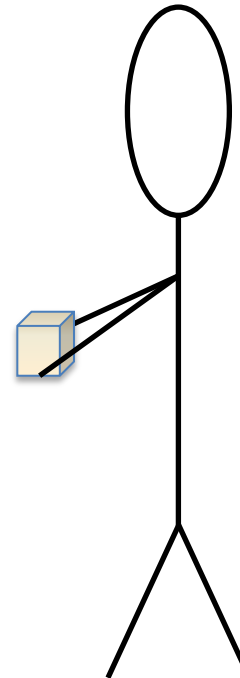
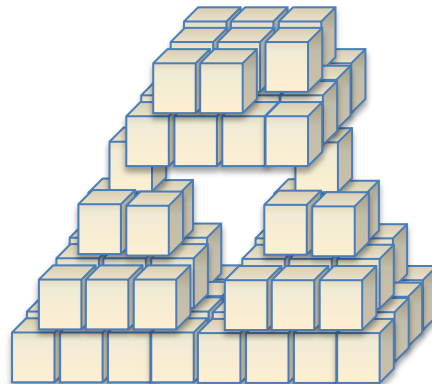
Optimizing a parallel program

- Sorting things:
 - Consider a pile of 90 boxes to sort
 - If it takes 10 seconds for one person to handle a box
 - It will take 900



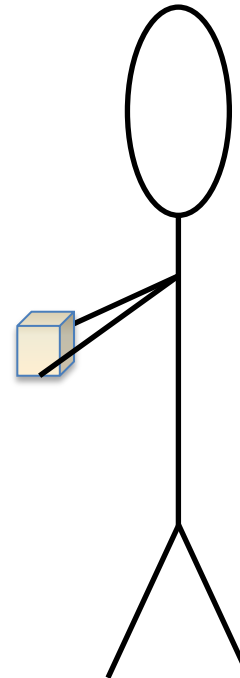
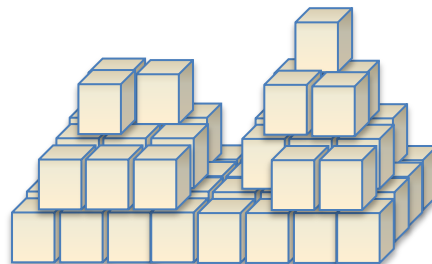
Optimizing a parallel program

- Sorting things:
 - Consider a pile of 90 boxes to sort
 - If it takes 10 seconds for one person to handle a box
 - It will take 900 seconds



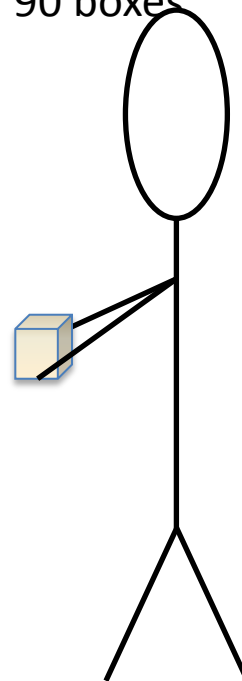
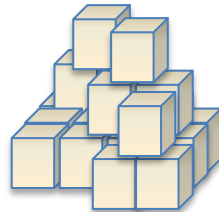
Optimizing a parallel program

- Sorting things:
 - Consider a pile of 90 boxes to sort
 - If it takes 10 seconds for one person to handle a box
 - It will take 900 seconds to handle all



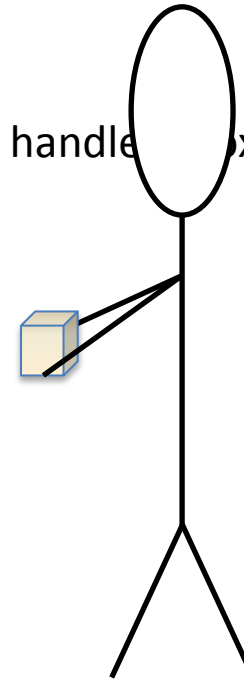
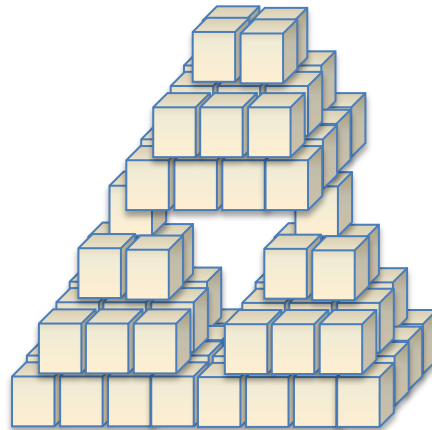
Optimizing a parallel program

- Sorting things:
 - Consider a pile of 90 boxes to sort
 - If it takes 10 seconds for one person to handle a box
 - It will take 900 seconds to handle all of the 90 boxes



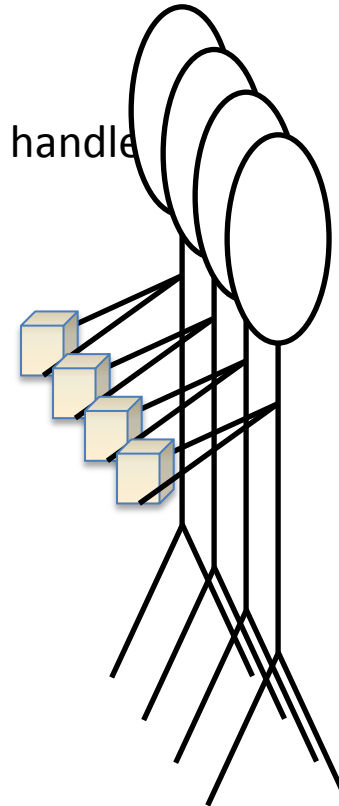
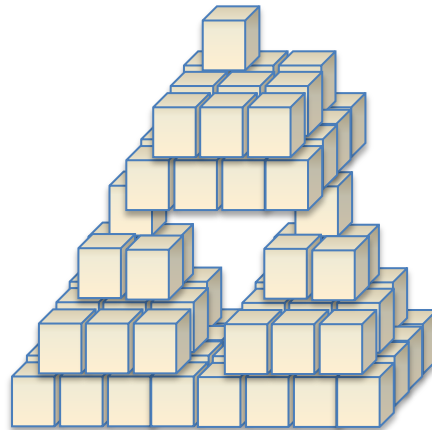
Optimizing a parallel program

- Sorting things:
 - Consider a pile of 90 boxes to sort
 - If it takes 10 seconds for one person to handle one box



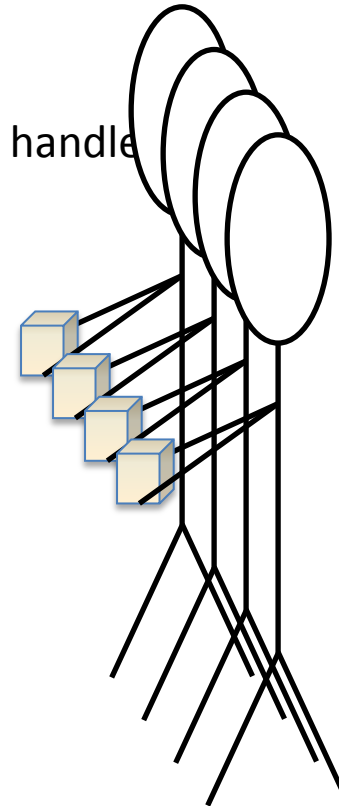
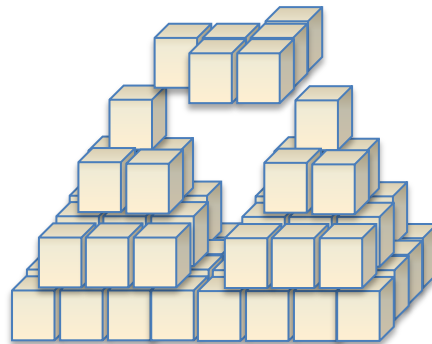
Optimizing a parallel program

- Sorting things:
 - Consider a pile of 90 boxes to sort
 - If it takes 10 seconds for one person to handle
 - And you ask four persons to do it



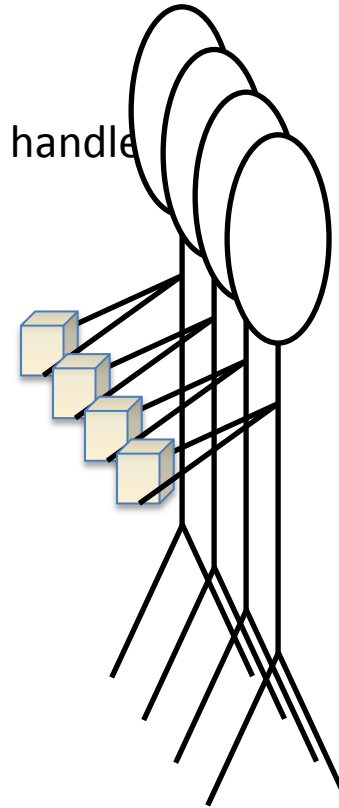
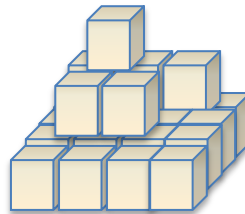
Optimizing a parallel program

- Sorting things:
 - Consider a pile of 90 boxes to sort
 - If it takes 10 seconds for one person to handle
 - And you ask four persons to do it
 - It will be four times faster



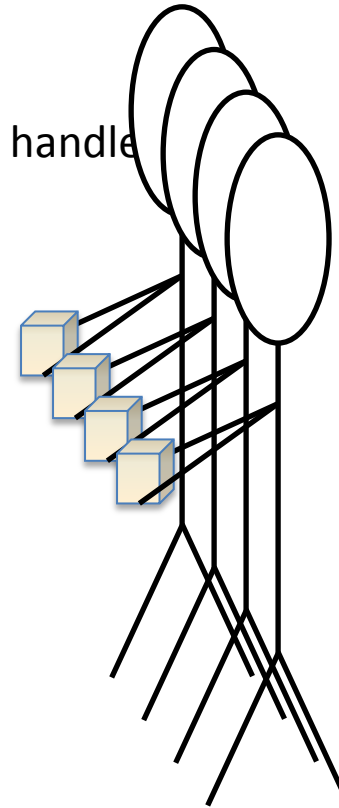
Optimizing a parallel program

- Sorting things:
 - Consider a pile of 90 boxes to sort
 - If it takes 10 seconds for one person to handle
 - And you ask four persons to do it
 - It will be four times faster
 - And be done in 225 seconds



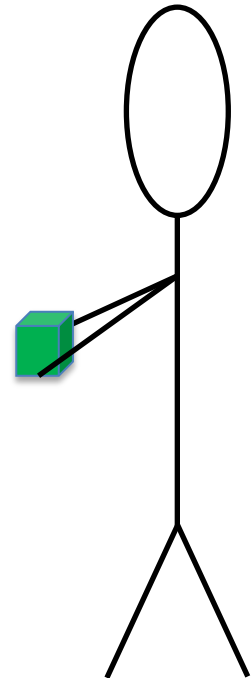
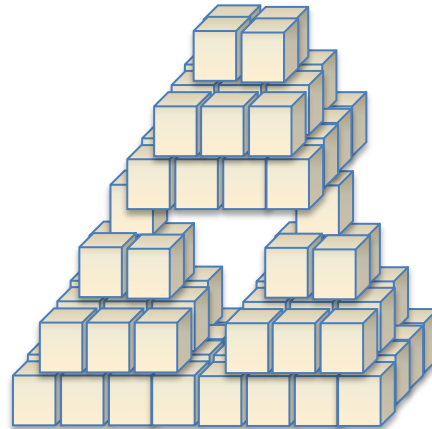
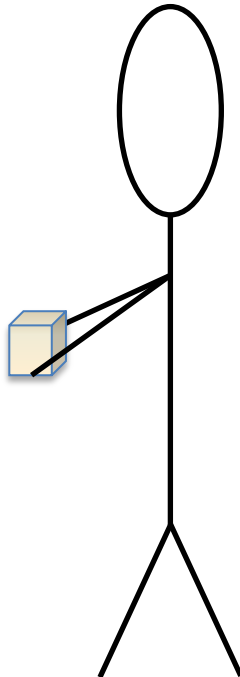
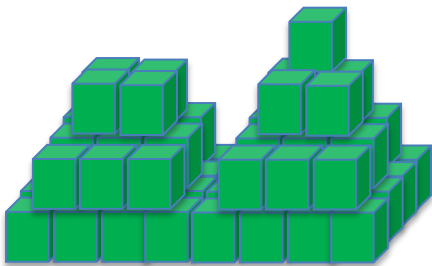
Optimizing a parallel program

- Sorting things:
 - Consider a pile of 90 boxes to sort
 - If it takes 10 seconds for one person to handle
 - And you ask four persons to do it
 - It will be four times faster
 - And be done in 225 seconds
- Of course that is the perfect case!
- But nothing in real life is perfect...



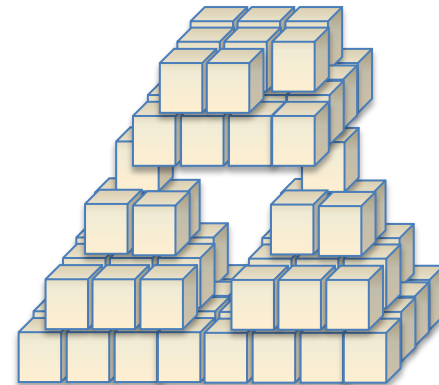
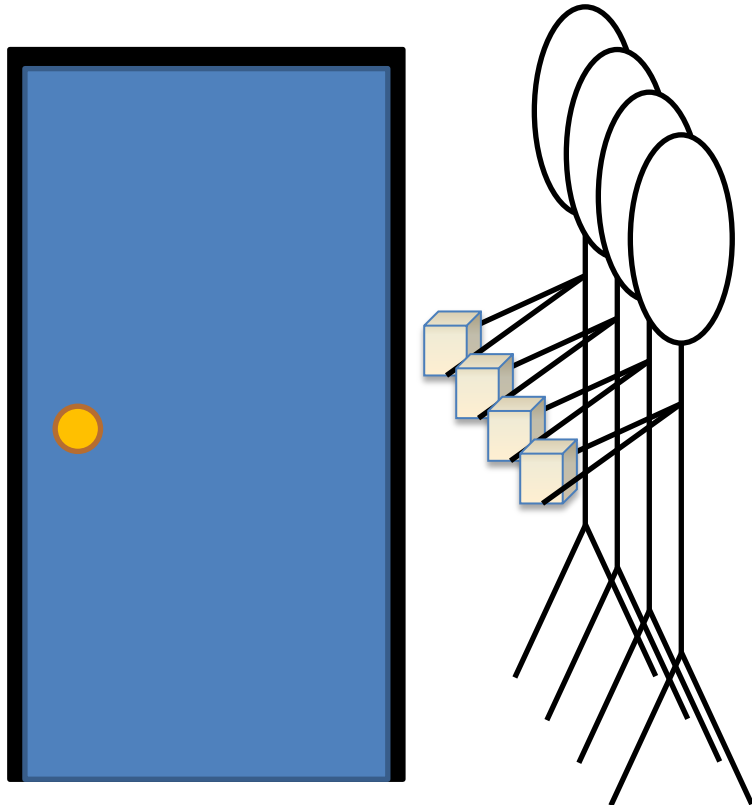
Optimizing a parallel program

- Sorting things:
 - What if one worker can only handle one type of work, and they don't have the same amount of work ?



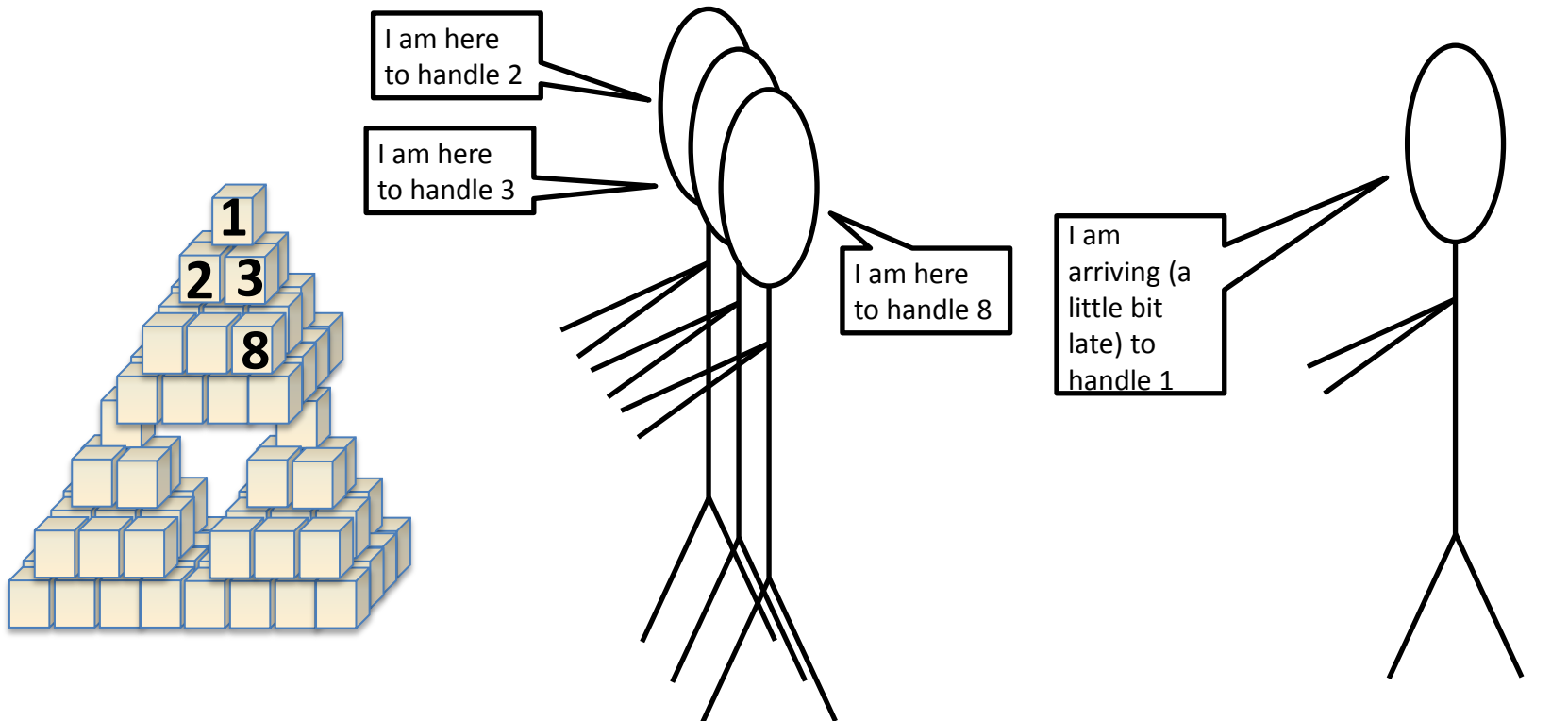
Optimizing a parallel program

- Sorting things:
 - What if all workers have to go through a single door to move the boxes ?



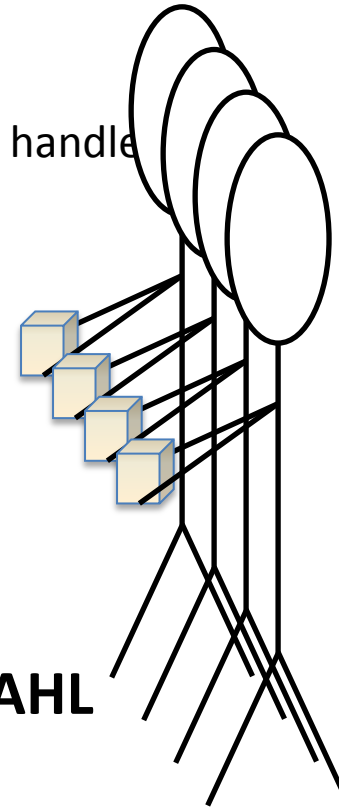
Optimizing a parallel program

- Sorting things:
 - What if the boxes have to be handled in a specific order, and each worker is specifically assigned to each box ?



Optimizing a parallel program

- Sorting things:
 - Consider a pile of 90 boxes to sort
 - If it takes 10 seconds for one person to handle
 - And you ask four men to do it
 - It will be four times faster
 - And be done in 225 seconds
- Of course that is the perfect case!
- But nothing in real life is perfect...
- ... and in parallelism, we have **AMDAHL**



AMDAHL's LAW

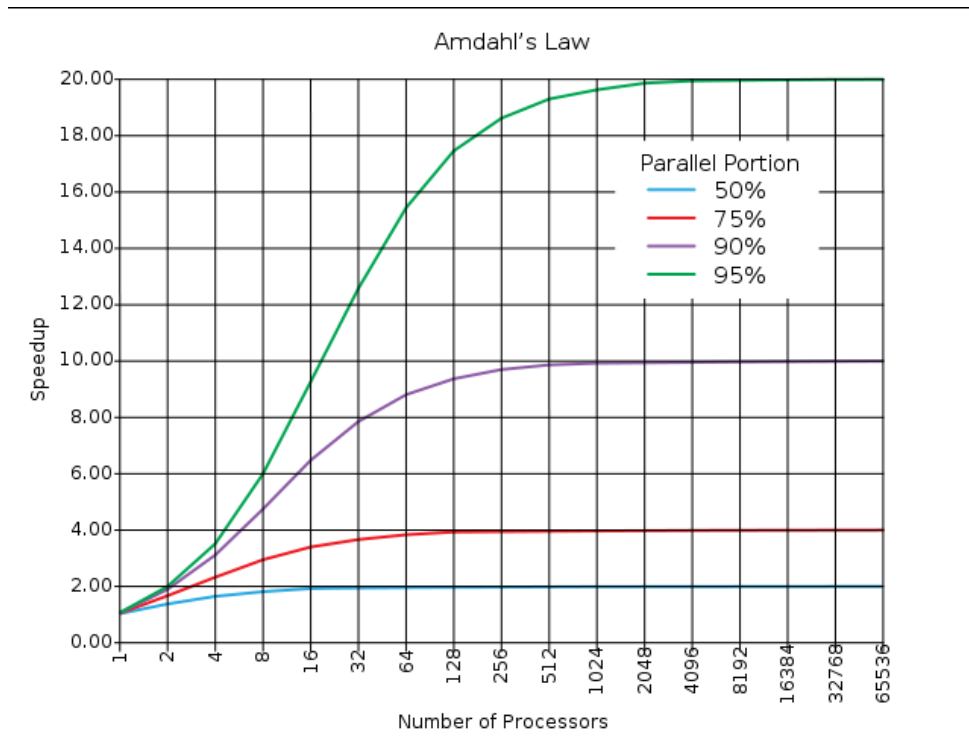


- Amdahl's law is very often use in parallel computing to assess the maximum speed-up a program can achieve in parallel
- It says that the maximum parallel speed-up one can achieve on a program is limited by the sequential part of this program
 - if a program is parallelized on 90% of its execution time, and the remain 10% is still sequential, then whatever the number of resources you will use for your program, you will never reduce the time under this threshold of 10%
 - Hence, the maximum speed-up you can achieve with parallelization if $x10$

AMDAHL's LAW illustration

- From wikipedia

https://en.wikipedia.org/wiki/Amdahl%27s_law



AMDAHL's LAW consequence

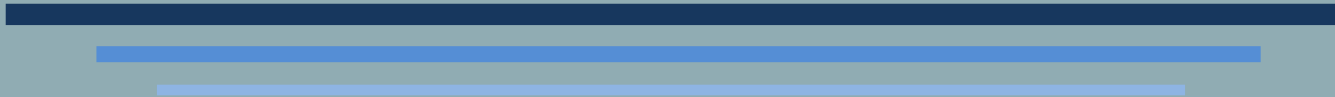


- What Amdahl's law means is:
 - Avoid sequential part in you programs
 - Avoid synchronization
 - If a worker W1 need to wait for another W2, it means that W1 won't do anything while waiting for W2
 - During this waiting time, only one worker (W2) will perform computation
 - This is the definition of sequential: only one worker works at a time
 - Synchronization -> sequential time -> performance loss

Studying scalability

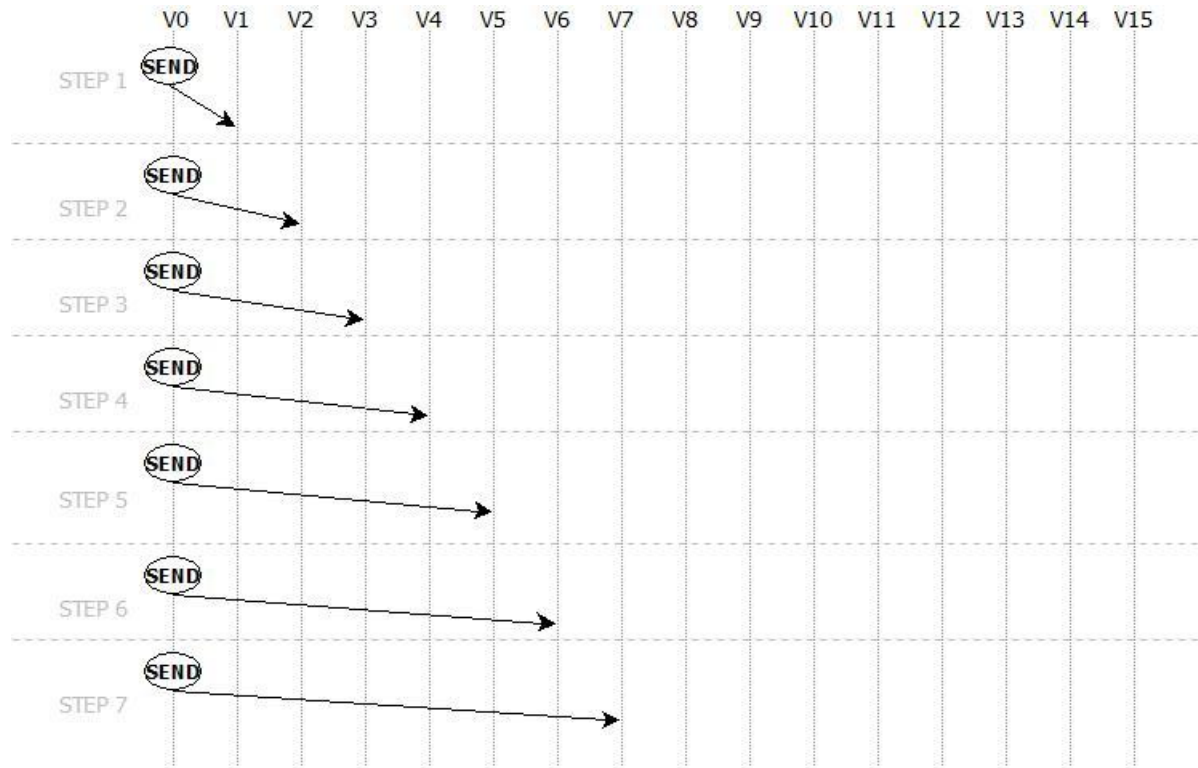
- The scalability of a parallel program is the evaluation of its behavior when one uses more workers to execute it
- A perfect scaling: double the resources -> twice as fast
- Scalability can be studied on two setups:
 - Strong scaling: how the performance evolves when using more resources for a *fixed total problem size*
 - See the sort box example
 - Double the resources, same global size, total time divided by 2
 - Weak scaling: how the performance evolves when using more resources for a *fixed problem size per resource*
 - Double the resources, double the global size, same time to handle twice as much computation/data/...

COLLECTIVE ALGORITHMS



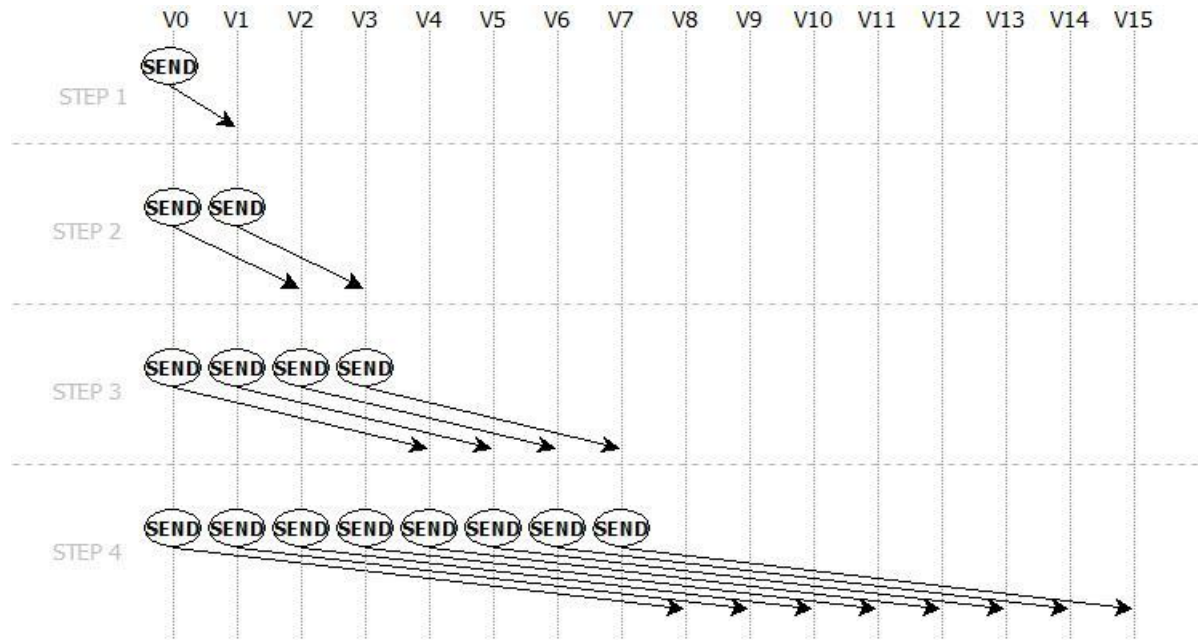
Broadcast algorithms

- Linear algorithm
- Root send its data to each other process
- Iterative and linear: for p processes, need $p-1$ step



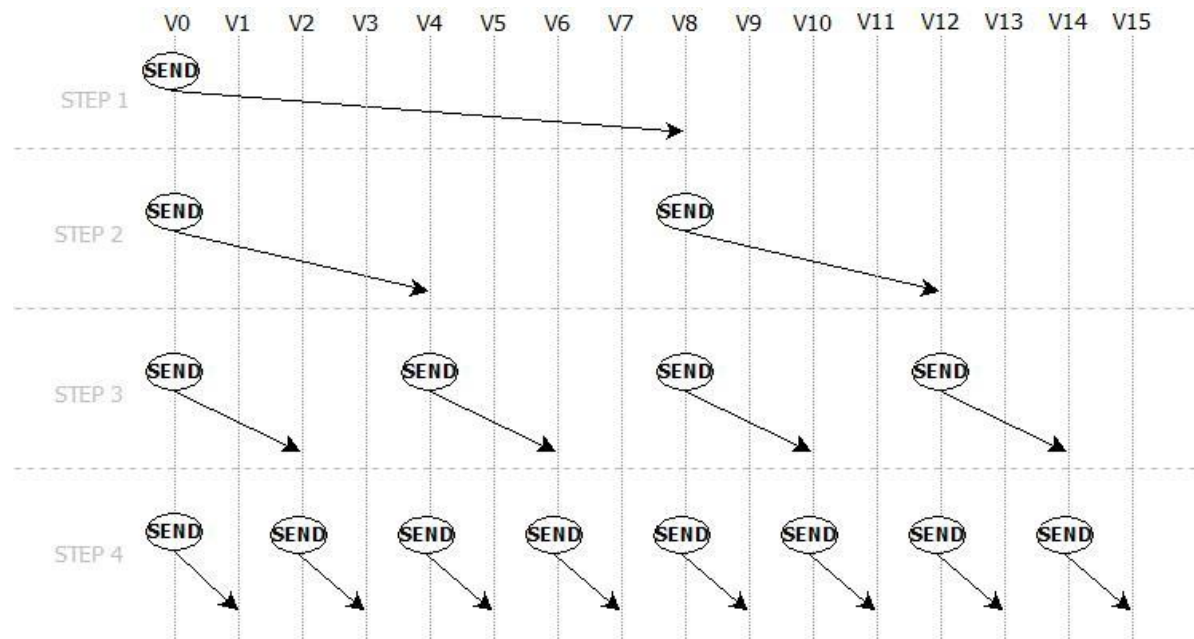
Broadcast algorithms

- Binary tree: at each step, each process with the data to broadcast send the data to another process
- Need $\log(p)$ steps
- `For(0; i<log(p); i++) if(rank<=i) send(data, rank+2i)`



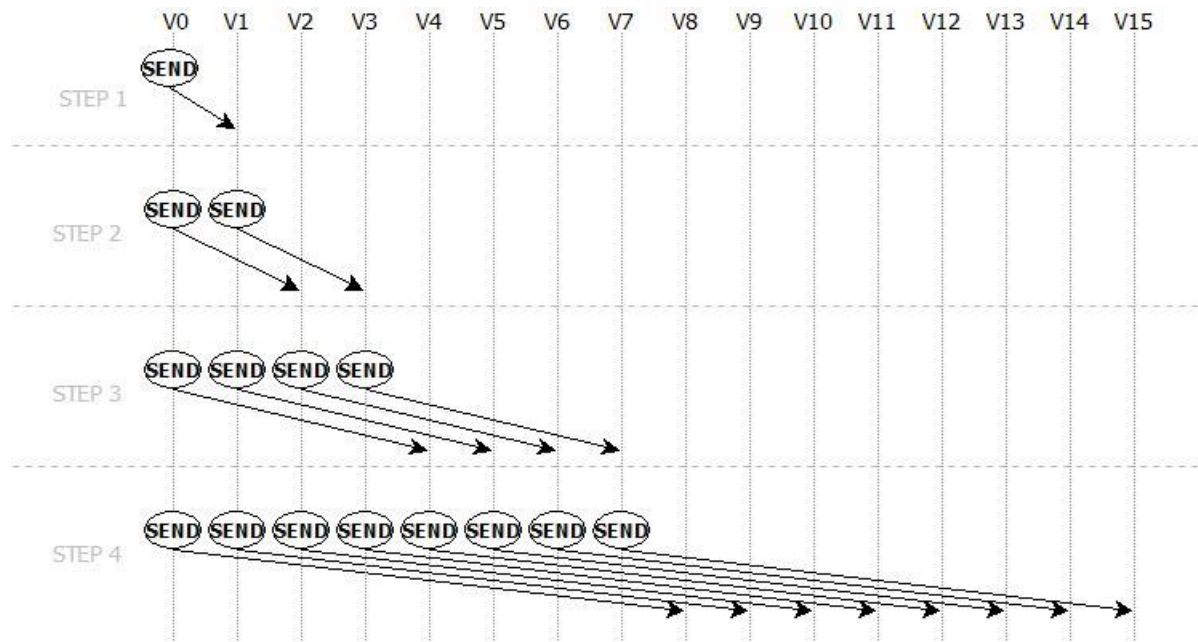
Broadcast algorithms

- Binary tree
- For($\log(p)$; $i > 0$; $i--$) if($\text{rank} \% 2^i == 0$) send(data, $\text{rank} + 2^{i-1}$)
- Are those two algorithms equivalent?



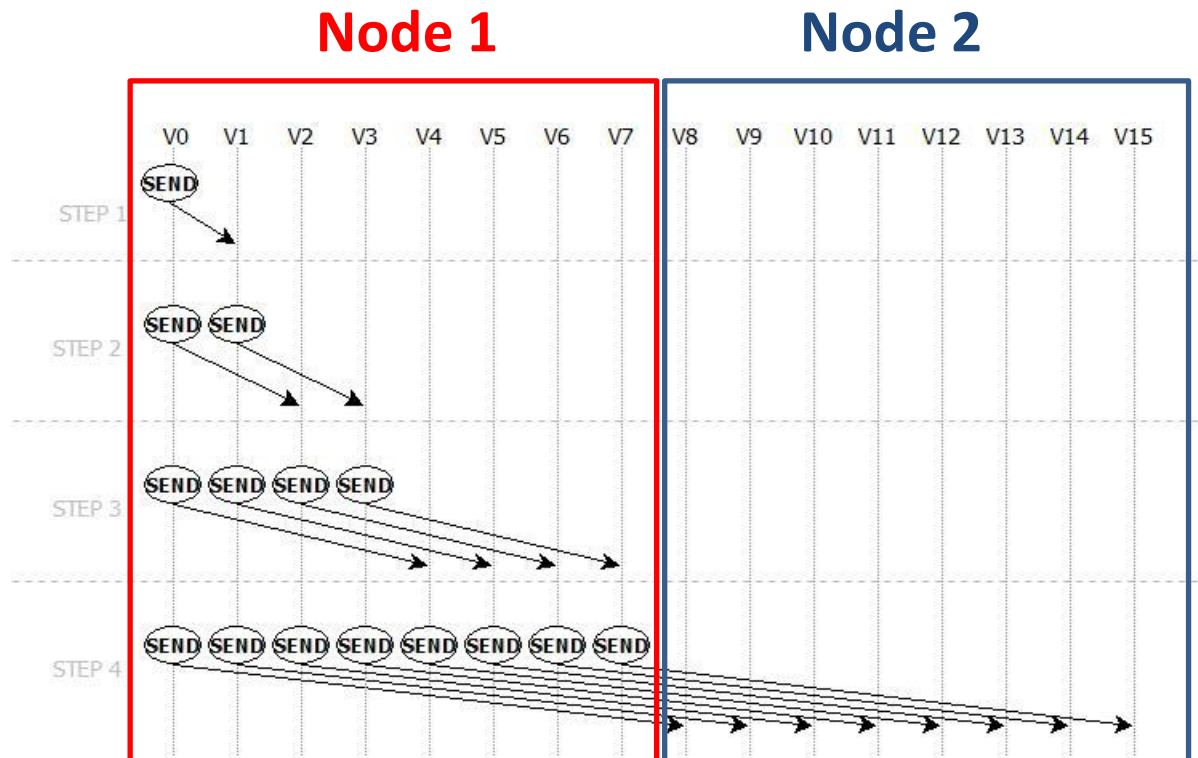
Broadcast algorithms

- If we consider two nodes



Broadcast algorithms

- If we consider two nodes

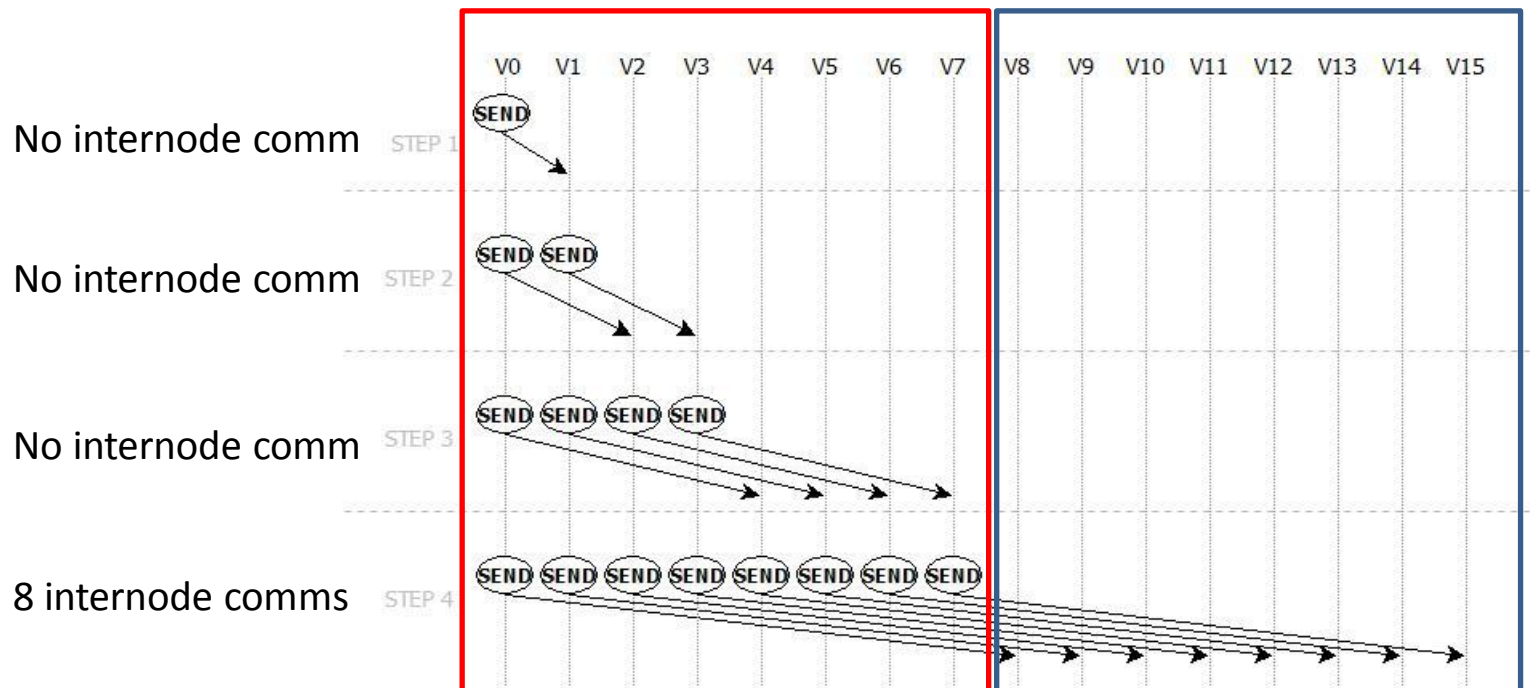


Broadcast algorithms

- If we consider two nodes
- All internode communications (network) happens at the same step

Node 1

Node 2



Broadcast algorithms

- Only one internode communication
- No contention on the network
- Intranode comm may be optimized

Node 1

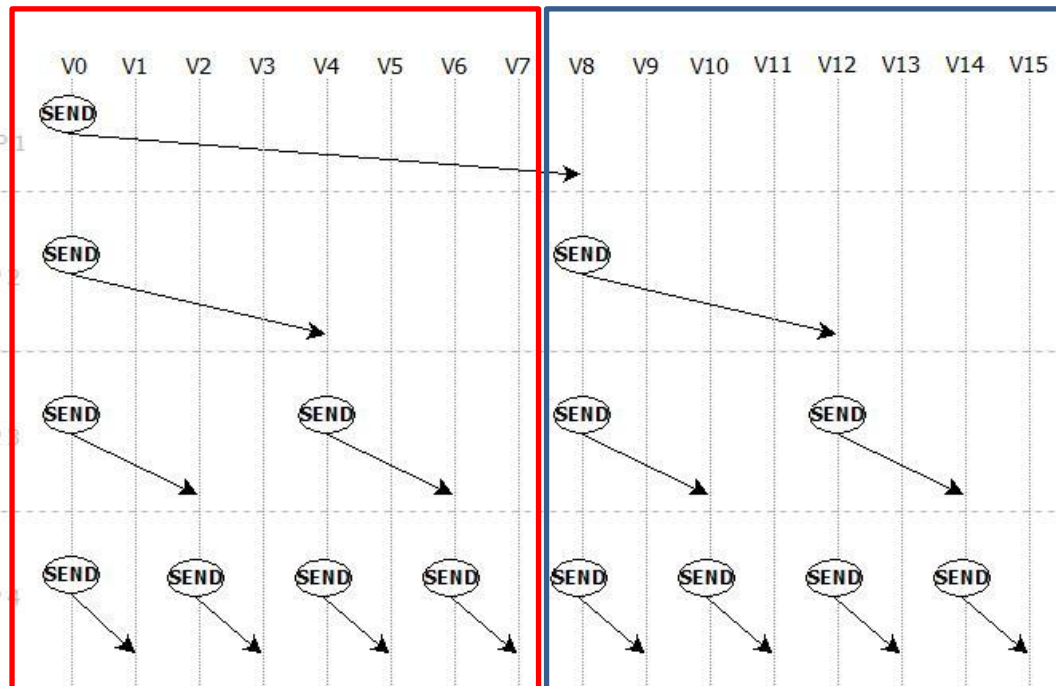
Node 2

1 internode comm

No internode comm

No internode comm

No internode comm

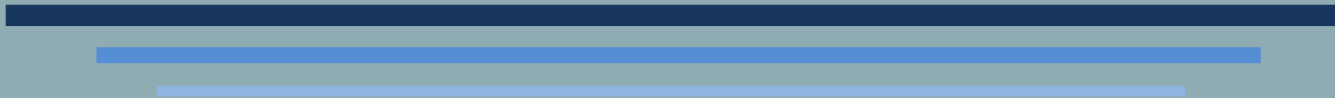


Collective algorithms



- Several algorithms may be implemented for one collective in a runtime
- Default selection of algorithms often depends on the size of the messages, and the number of MPI processes involved
- Algorithm selection may also be available to user through launch options, environment variables or communicator attributes

NONBLOCKING COMMUNICATIONS

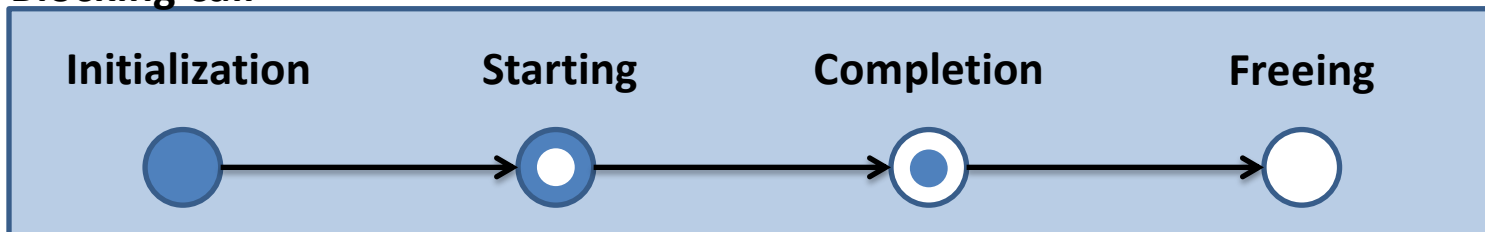


Blocking Communications

■ Definition

- A **procedure is blocking** if return from the procedure indicates the user is allowed to reuse resources specified in the call.
- For a **blocking operation**, all four stages are combined in a single blocking procedure call.

Blocking call



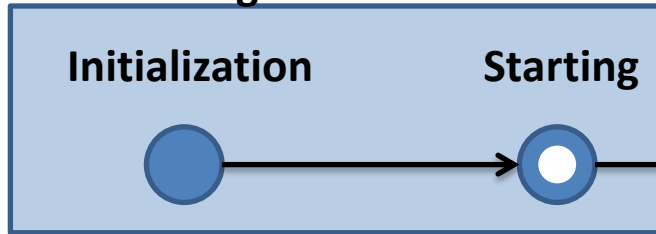
Blocking operation

Nonblocking Communication

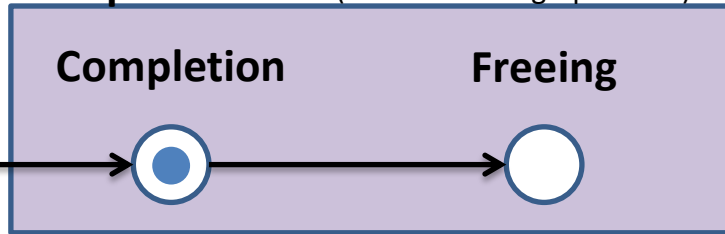
■ Definition

- For a nonblocking operation, the initialization and starting stages are combined into a single nonblocking procedure call and the completion and freeing stages are combined into a separate, single procedure call, which can be blocking or nonblocking

Nonblocking call



Completion call (for nonblocking operation)



Nonblocking operation

■ Meaning

- No safe access to input message when function send returns
- To be sure that message buffer can be reused, an additional function should be called and returned
- IT IS FORBIDDEN TO MODIFY THE BUFFER BETWEEN THE TWO CALLS

Nonblocking Send MPI_Isend



```
int MPI_Isend (  
void *buf(in),  
int count(in),  
MPI_Datatype datatype(in),  
int dest(in),  
int tag(in),  
MPI_Comm comm(in),  
MPI_Request *req(out)  
);
```

One additional argument
MPI_Request *req.

Request id is returned in
*req (MPI_Request = MPI
opaque type).

To finish the communication
a completion procedure
should be called.

Check Function MPI_Wait

```
int MPI_Wait (  
    MPI_Request *req(inout),  
    MPI_Status *sta(out)  
);
```

MPI_Wait blocks until operation represented by *req is completed.

Detailed information about finished communication are store into *sta.

When MPI_Wait returns

- *req is assigned to MPI_REQUEST_NULL (invalid request)
- Input message buffer can be safely manipulated by sender

Remark:

$\text{MPI_Send} \approx \text{MPI_Isend} + \text{MPI_Wait}$

Nonblocking Example

Instructions
between
`MPI_Isend` and
`MPI_Wait` should
not write into `buf`.

```
MPI_Request req;  
MPI_Status sta;
```

```
MPI_Isend(buf, N, MPI_BYTE,  
dest, tag1, comm,  
&req);
```

```
instruction1;  
instruction2;  
...  
instructionN;
```

```
MPI_Wait(&req, &sta);
```

In the meantime,
message progresses

■ Advantages

- Recover communications and computation

Nonblocking Communication

```
int MPI_Test (  
MPI_Request *req(inout),  
int *flag(out),  
MPI_Status *sta(out)  
);
```

Write true (non-zero value) in *flag if request *req is over.

If *flag is true, *req is assigned to MPI_REQUEST_NULL and *sta is filled.

If *flag is false, values of *req and *sta are not guaranteed.

Nonblocking Communication



■ Example :

```
MPI_Irecv(msg, N, MPI_BYTE, dest, tag, comm, &req);  
do {  
    instruction1;  
    ...  
    instructionN;  
  
    MPI_Test(&req, &flag, &sta);  
} while( !flag );
```

Nonblocking Communication

```
int MPI_Waitall (  
    int nb_req(in),  
    MPI_Request *tab_req(inout),  
    MPI_Status *tab_sta(out)  
);
```

Return when `nb_req` requests located in array `tab_req` are completed.

Status of communications are available as output in array `tab_sta`.

Remark:

Order of communication completion is not important



Nonblocking Communication



- Example: send/receive with left/right neighbors

```
MPI_Request req[4];
MPI_Status sta[4];

left = (rang + P - 1) % P;
right = (rang + 1) % P;

MPI_Isend(&x[1], 1, MPI_DOUBLE, left, tag, comm, req);
MPI_Isend(&x[N], 1, MPI_DOUBLE, right, tag, comm, req+1);
MPI_Irecv(&x[0], 1, MPI_DOUBLE, left, tag, comm, req+2);
MPI_Irecv(&x[N+1], 1, MPI_DOUBLE, right, tag, comm, req+3);

MPI_Waitall(4, req, sta);
```

Other Available Functions



- MPI proposes multiple functions to complete non-blocking communications
- `MPI_Testall`
 - Test is all requests as input are completed
- `MPI_Waitany / MPI_Testany`
 - Wait/Test until at least one request is completed
 - Return index of completed request
- `MPI_Waitsome / MPI_Testsome`
 - Wait/Test until at least one request is completed
 - Return set of completed requests

Communications and modes

- Nonblocking communication is different from asynchronous
- Nonblocking communications can be done in different modes: synchronous, buffered or regular

Type/Mode	Standard	Buffered	Synchronous	Receive
Blocking	MPI_Send	MPI_Bsend	MPI_Ssend	MPI_recv
Non-Blocking	MPI_Isend	MPI_Ibsend	MPI_Issend	MPI_Irecv

MPI nonblocking Collectives



- Provide all MPI collectives communications with nonblocking semantics
- Goal: get the best of collective operations...
 - Optimized communications patterns
- ... and nonblocking communications
 - Allows to overlap communication with computation
 - More communications involved in a collective call than p2p
 - => more opportunities to gain performances from overlapping
- Same semantic as nonblocking point-to-point
 - nonblocking communication calls initiate the communication
 - Operation is not finished upon return from communication call
 - Need a completion call to ensure input buffered can be reused
 - Wait* and Test* calls
 - Function name is the same but begins with "I"

Same rules as blocking collectives



- All processes in the group identified by the communicator must call the nonblocking collective routine
- Inside a communicator, processes should call the same sequence of collective communication (**blocking and nonblocking**)
- Between two ranks in different communicators, there are no restrictions

Same behavior as blocking collectives



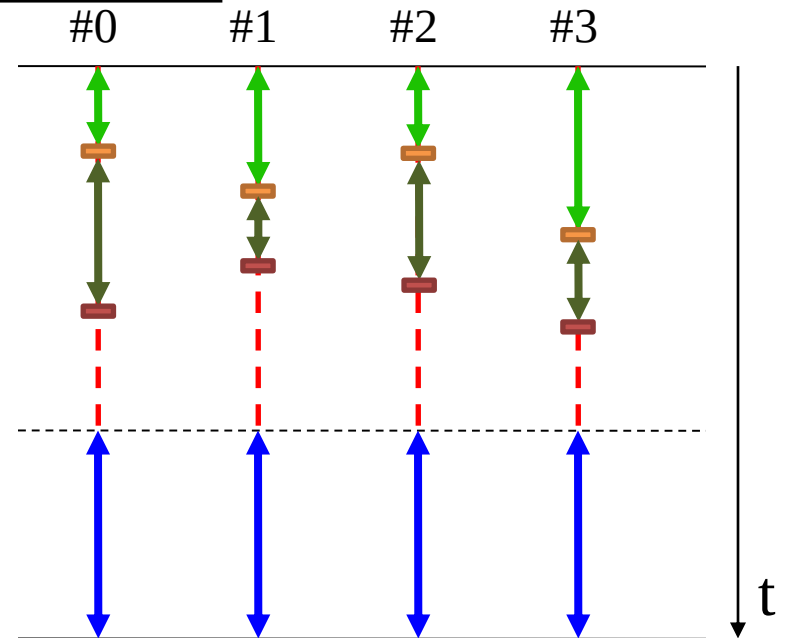
- Completing a nonblocking collective call (e.g. after the matching completion call) on a process does not mean that the collective is done for all processes
 - It just means that the contribution of the current process is done
- If the current process does not require the result but only send data, it basically is the same as `MPI_Isend`
 - After the completion call, the user can reuse the input buffer
 - It does not even mean that the data has been send
- Same function signature, adding an *MPI_Request* argument

nonblocking Barrier

- Synchronize all processes belonging to target communicator

```
int MPI Ibarrier( MPI Comm comm,  
                  MPI Request * req ) ;
```

```
MPI_Init(&argc, &argv);  
MPI_Request req;  
/* Work 1 */  
MPI_Ibarrier(MPI_COMM_WORLD,  
             &req);  
/* Work 2 */  
  
MPI_Wait(req, status);  
  
/* Work 2 */  
  
MPI_Finalize();
```



nonblocking Broadcast



```
int MPI_Ibcast (  
    void *buf(inout),  
    int count(in),  
    MPI_Datatype datatype(in),  
    int root(in),  
    MPI_Comm comm(in),  
    MPI_Request *req(inout)  
);
```

Request `req` is update with nonblocking calls information to give to completion call.

nonblocking Scatter



```
int MPI_Iscatter (  
    void *sendbuf(in),  
    int sendcount(in),  
    MPI_Datatype sendtype(in),  
    void *recvbuf(out),  
    int recvcount(in),  
    MPI_Datatype recvtype(in),  
    int root(in),  
    MPI_Comm comm(in),  
    MPI_Request *req(inout)  
);
```

Request `req` is update with
nonblocking calls information to give
to completion call.

nonblocking Gather



```
int MPI_Igather (  
    void *sendbuf(in),  
    int sendcount(in),  
    MPI_Datatype sendtype(in),  
    void *recvbuf(out),  
    int recvcount(in),  
    MPI_Datatype recvtype(in),  
    int root(in),  
    MPI_Comm comm(in),  
    MPI_Request *req(inout)  
);
```

Request `req` is update with
nonblocking calls information to give
to completion call.

nonblocking Allgather



```
int MPI_Iallgather (  
    void *sendbuf(in),  
    int sendcount(in),  
    MPI_Datatype sendtype(in),  
    void *recvbuf(out),  
    int recvcount(in),  
    MPI_Datatype recvtype(in),  
    MPI_Comm comm(in),  
    MPI_Request *req(inout)  
);
```

Request `req` is update with
nonblocking calls information to give
to completion call.

Alltoall



```
int MPI_Ialltoall (  
    void *sendbuf(in),  
    int sendcount(in),  
    MPI_Datatype sendtype(in),  
    void *recvbuf(out),  
    int recvcount(in),  
    MPI_Datatype recvtype(in),  
    MPI_Comm comm(in),  
    MPI_Request *req(inout)  
);
```

Request `req` is update with
nonblocking calls information to give
to completion call.

nonblocking Reduction



```
int MPI_Ireduce (  
    void *sendbuf(in),  
    void *recvbuf(out),  
    int count(in),  
    MPI_Datatype datatype(in),  
    MPI_Op op(in),  
    int root(in),  
    MPI_Comm comm(in),  
    MPI_Request *req(inout)  
);
```

Request `req` is update with
nonblocking calls information to give
to completion call.

nonblocking Reduce to All



```
int MPI_Iallreduce (  
    void *sendbuf(in),  
    void *recvbuf(out),  
    int count(in),  
    MPI_Datatype datatype(in),  
    MPI_Op op(in),  
    MPI_Comm comm(in),  
    MPI_Request *req(inout)  
);
```

Request `req` is update with
nonblocking calls information to give
to completion call.

nonblocking Partial Reduction



```
int MPI_Iscan (  
    void *sendbuf(in),  
    void *recvbuf(out),  
    int count(in),  
    MPI_Datatype datatype(in),  
    MPI_Op op(in),  
    int root(in),  
    MPI_Comm comm(in),  
    MPI_Request *req(inout)  
);
```

Request `req` is update with
nonblocking calls information to give
to completion call.

nonblocking MPI_Scatterv

```
int MPI_Iscatterv (  
    void *sendbuf(in),  
    int *sendcounts(in),  
    int *displs(in),  
    MPI_Datatype sendtype(in),  
    void *recvbuf(out),  
    int recvcount(in),  
    MPI_Datatype recvtype(in),  
    int root(in),  
    MPI_Comm comm(in),  
    MPI_Request *req(inout)  
);
```

Request `req` is update with nonblocking calls information to give to completion call.

nonblocking MPI_Alltoallw

```
int MPI_Ialltoallw (  
    void *sendbuf(in),  
    int *sendcounts(in),  
    int *displs(in),  
    MPI_Datatype *sendtypes(in),  
    void *recvbuf(out),  
    int *recvcounts(in),  
    MPI_Datatype *recvtypes(in),  
    int root(in),  
    MPI_Comm comm(in),  
    MPI_Request *req(inout)  
);
```

Request `req` is update with nonblocking calls information to give to completion call.

nonblocking Reduction-Scatter block



```
int MPI_Ireduce_Scatter_block (  
    void *sendbuf(in),  
    void *recvbuf(out),  
    int recvcount(in),  
    MPI_Datatype datatype(in),  
    MPI_Op op(in),  
    MPI_Comm comm(in),  
    MPI_Request *req(inout)  
);
```

Request `req` is update with
nonblocking calls information to give
to completion call.

nonblocking Partial Reduction



```
int MPI_Iexscan (  
    void *sendbuf(in),  
    void *recvbuf(out),  
    int count(in),  
    MPI_Datatype datatype(in),  
    MPI_Op op(in),  
    int root(in),  
    MPI_Comm comm(in),  
    MPI_Request *req(inout)  
);
```

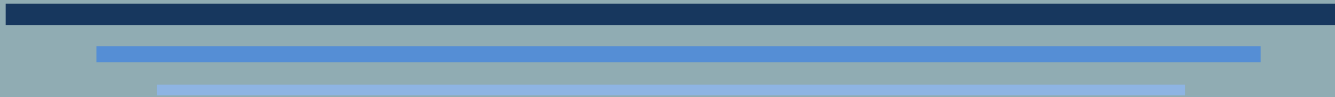
Request `req` is update with
nonblocking calls information to give
to completion call.

MPI nonblocking Collectives



- To obtain performance with NBC
 - Start communication as soon as possible
 - Insert completion call as late as possible
 - Maximize the overlap potential between communication and compute
- Depends on NBC implementation
 - If no progression thread, difficult to overlap
 - Still possible with a NIC handling message passing
 - With progression thread
 - Where? All cores may be used for MPI, or other models (threads...)
 - On the same node: may slow down computation
 - On another node: may slowdown another MPI process, or another thread
 - Allocate specific resources for progression thread: is it worth it?
 - Less communications but larger communication buffers and larger compute phase per MPI process

PERSISTENT COMMUNICATIONS

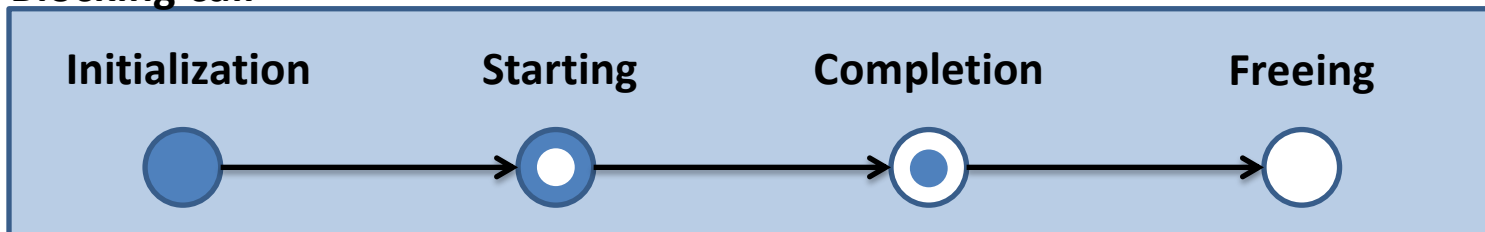


Blocking Communications

■ Definition

- A **procedure is blocking** if return from the procedure indicates the user is allowed to reuse resources specified in the call.
- For a **blocking operation**, all four stages are combined in a single blocking procedure call.

Blocking call



Blocking operation

Nonblocking Communication

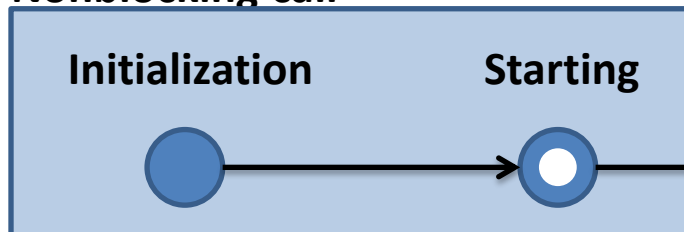
- Definition

- For a **nonblocking operation**, the initialization and starting stages are combined into a single nonblocking procedure call and the completion and freeing stages are combined into a separate, single procedure call, which can be blocking or nonblocking

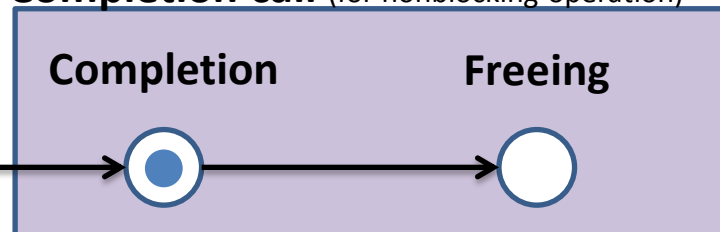
- Meaning

- IT IS FORBIDDEN TO MODIFY THE BUFFER BETWEEN THE TWO CALLS
- To be sure that message buffer can be reused, an additional function should be called and returned

Nonblocking call



Completion call (for nonblocking operation)

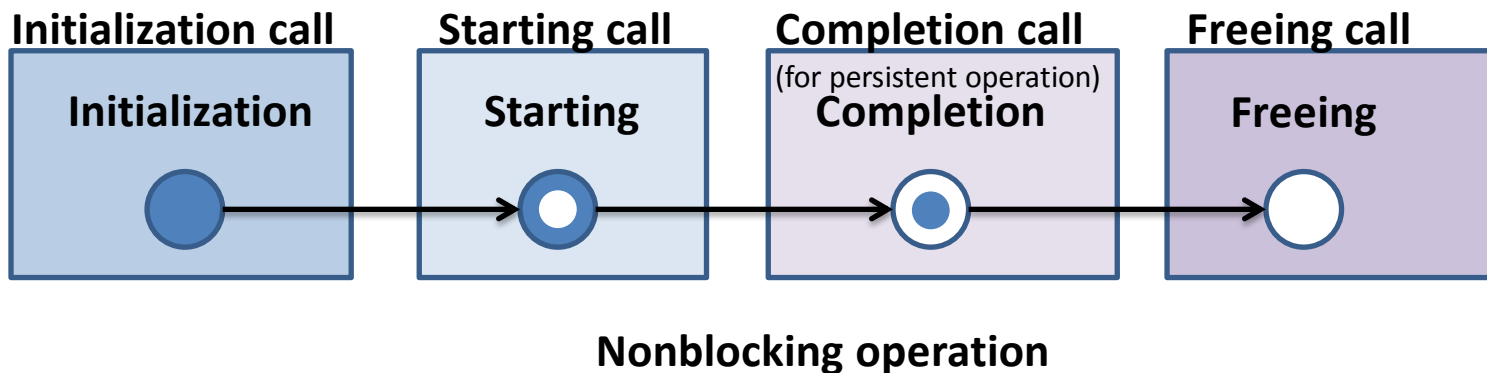


Nonblocking operation

Persistent Communication

■ Definition

- For a **persistent operation**, there is a separate procedure for each of the four stages. Each of these procedures may be blocking or nonblocking.



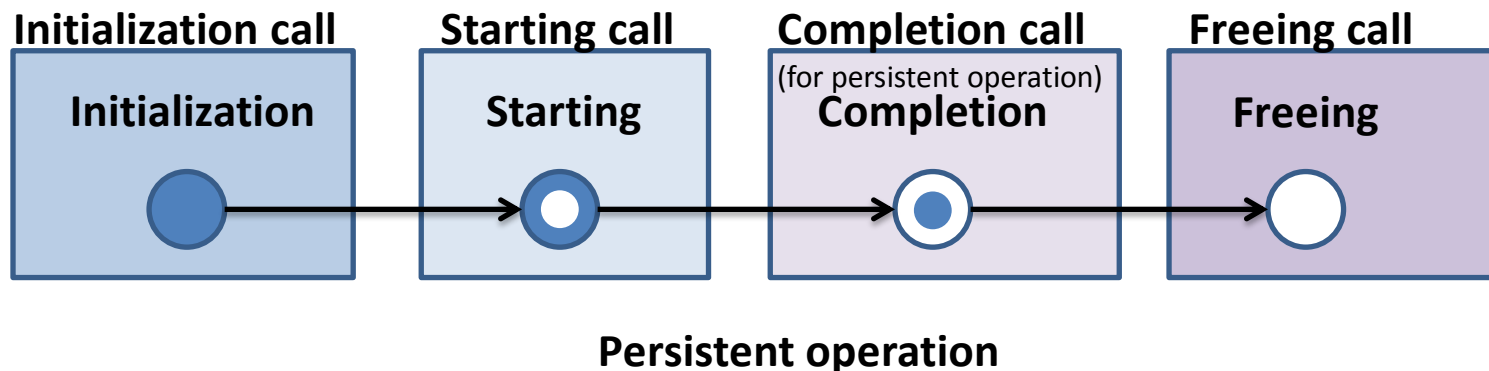
Persistent Communication

■ Definition

- For a **persistent operation**, there is a separate procedure for each of the four stages. Each of these procedures may be blocking or nonblocking.

■ Allows more buffer access

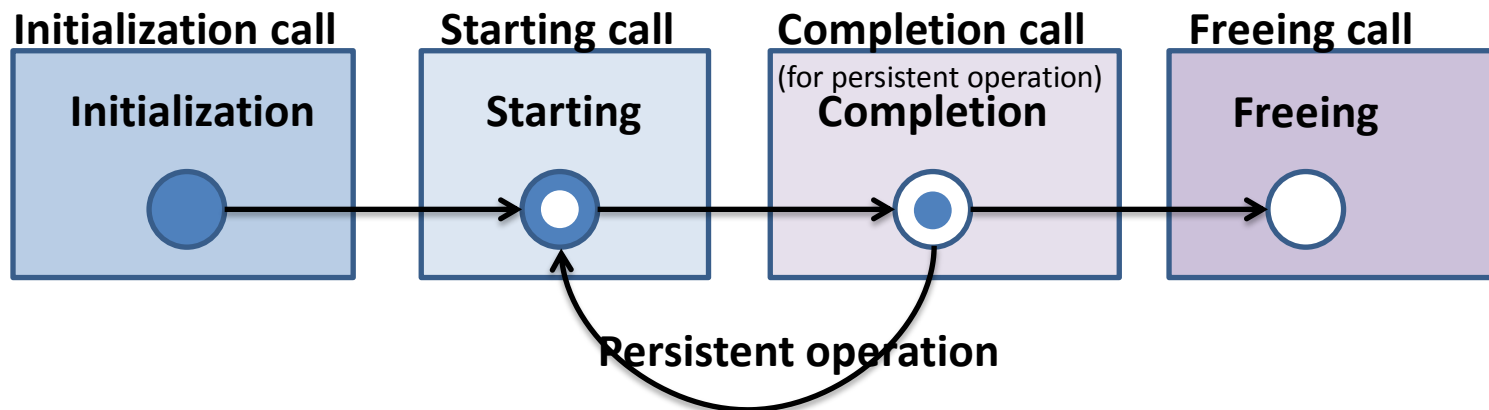
- Buffer can be safely accessed between init and start
- Buffer should not be accessed between start and completion
- Buffer can be safely accessed between completion and free



Persistent Communication

■ Definition

- For a **persistent operation**, there is a separate procedure for each of the four stages. Each of these procedures may be blocking or nonblocking.
- Allows to redo the same operation (hence persistent)
 - Only one init
 - Exact same parameters
 - Can modify input buffers (before start and after completion)



Persistent Send MPI_Send_Init



```
int MPI_Send_Init (  
void *buf(in),  
int count(in),  
MPI_Datatype datatype(in),  
int dest(in),  
int tag(in),  
MPI_Comm comm(in),  
MPI_Request *req(out)  
);
```

Same signature as
MPI_Isend

Start Function MPI_Start

```
int MPI_Start (  
    MPI_Request *req(inout),  
);
```

MPI_Start starts the persistent operation represented by the request *req. It is forbidden to modify the buffer of the operation after MPI_Start returns.

It is possible to start multiple request with MPI_Start_all. The MPI_Start_all procedure takes an array of requests to be started

```
int MPI_Start_all (  
    Int count ,  
    MPI_Requests *array_of_reqs(inout),  
);
```

Completion procedure



- After starting, a persistent operation must be completed
- Any completion calls used for nonblocking operations can be used
 - `MPI_Wait` / all / any / some
 - `MPI_Test` /all / any / some
- `/!\` For a persistent operation, the completion procedure does not free the request (hence the operation)
 - An additional procedure should be called
- This is why it is safe to call again “`MPI_Start`” once a persistent operation has been completed

Start Function MPI_Start

```
int MPI_Request_free (  
    MPI_Request *req(inout),  
);
```

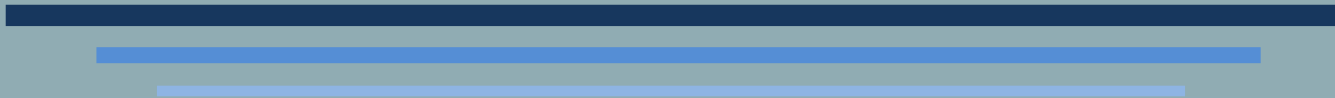
MPI_Request_free frees the persistent operation represented by the request *req.

It is not possible anymore to call the starting procedure on this request.

It is not possible anymore to check for completion with completion procedure calls.

There is no multiple requests freeing procedure (such as MPI_Request_free)

SUPERCOMPUTERS OVERVIEW



Classification and french ecosystem

Supercomputer Classification

- Small application to compare machines

- Benchmark or *miniapp* or *proxyapp*
- Results → metrics able to compare machines

- Example: **Top500**

- Rank machines according to the computational power on regular codes
- Homepage: <http://www.top500.org>

- Benchmark: Linpack

- Linear solver based on linear algebra
- Relies on performance of DGEMM
- Towards HPCG (Conjugate Gradient)



Top500

- List of 500 most powerful machines
 - Measure mainly the computational power
 - According to Linpack results

- Updated twice a year
 - June: ISC conference in Germany
 - November: SC conference in US

- Machine Information
 - Main info (rank, site)
 - System (name and short description)
 - Number of cores
 - Performance (R_{max} , R_{peak})
 - Power

- Notes
 - Performance in Pflops/s (10^{15} floating-point operations per second)
 - Difference between max performance (R_{max}) and Linpack result (R_{peak})
 - Power measured in kW



Top500 (#1 to #5)

Rank	Country	System	Cores	Rmax	Rpeak	Power
1	United States	Frontier	8,730,112	1,102.00	1,685.65	21,100
2	Japan	Fugaku	7,630,848	442.01	537.21	29,899
3	Finland	Lumi	1,110,144	151.90	214.35	2,942
4	United States	Summit	2,414,592	148.60	200.79	10,096
5	United States	Sierra	1,572,480	94.64	125.71	7,438



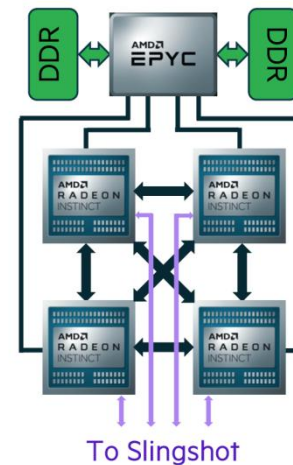
Top500 (#6 to #10)

Rank	Country	System	Cores	Rmax	Rpeak	Power
6	China	Sunway TaihuLight	10,649,600	93.01	125.44	15,371
7	United States	Perlmutter	761,856	70.87	93.75	2,589
8	United States	Selene	555,520	63.46	79.22	2,646
9	China	Tianhe-2	4,981,760	61.44	100.68	18,482
10	France	Adastr	319,072	46.10	61.61	921



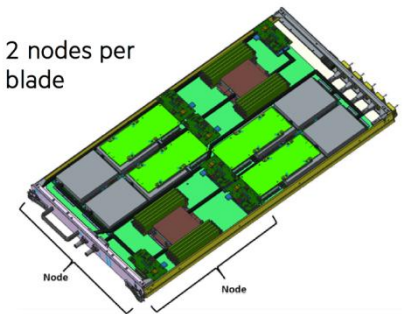
Top500 #1

- HPE Frontier (at Oak Ridge National Lab - ORNL)
- Total: 8,730,112 cores
 - 74 armoires
 - AMD Epyc 3rd gen
 - 1 cpu + 4 Instinct GPUs / node
 - HPE Slingshot interconnect



AMD GPU
(ORNL)

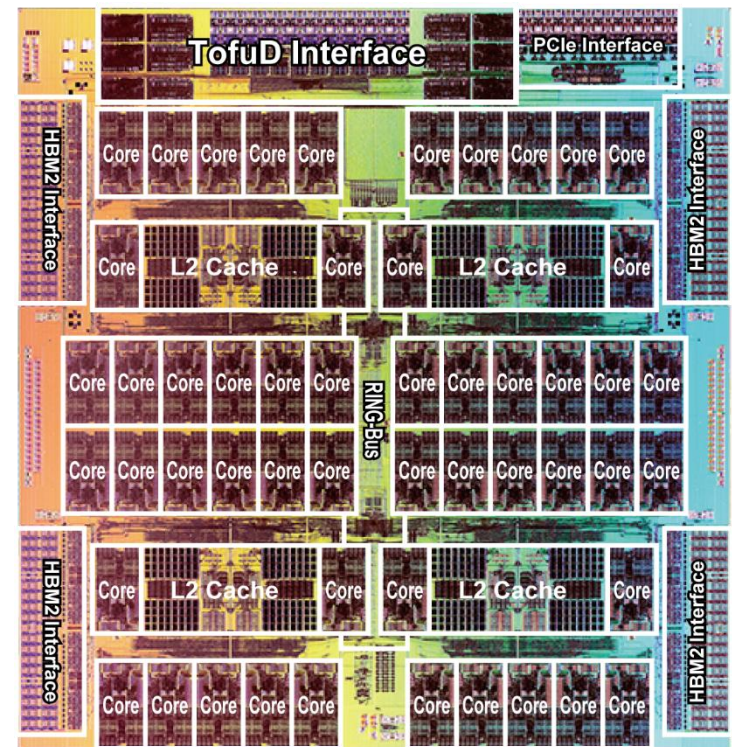
2 nodes per
blade



COPYRIGHT 2020 HPE

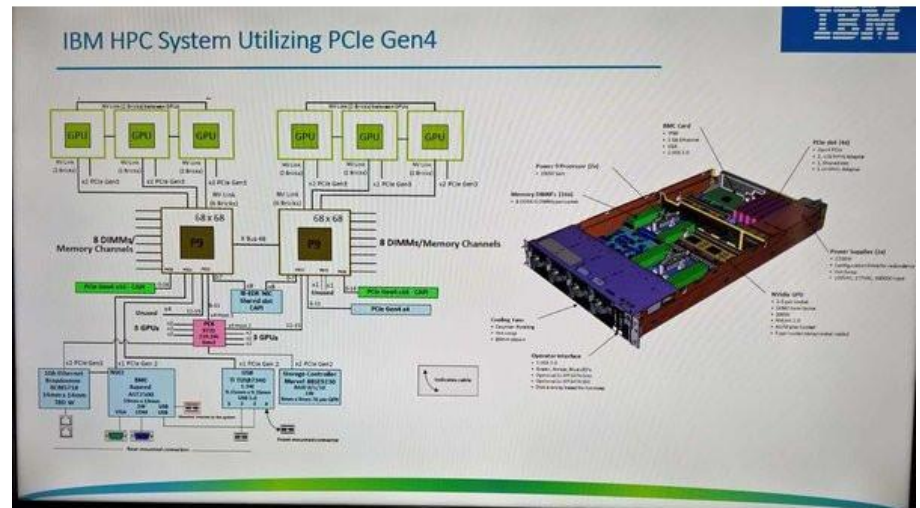
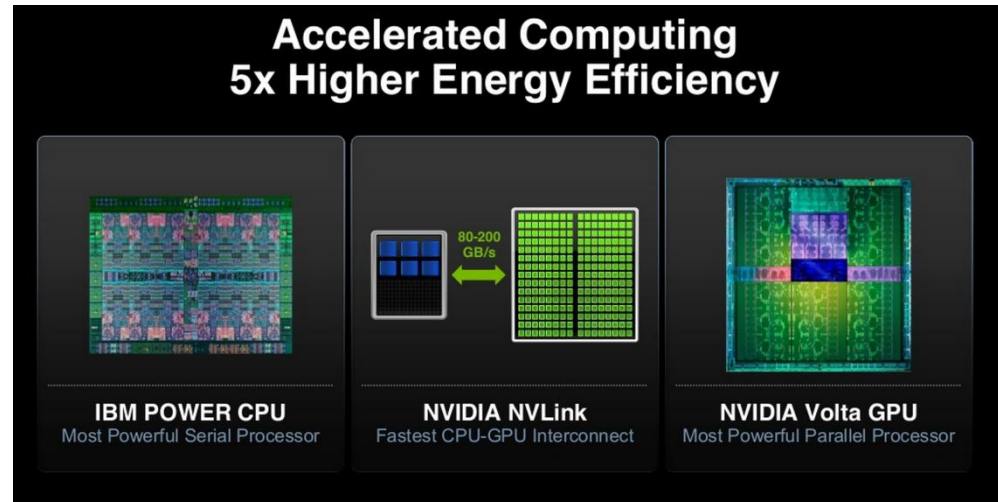
Top500 #2

- Fujitsu FUGAKU (at RIKEN)
- Total: 7,299,072 cores
 - 158,976 nodes
 - Armv8.2-A SVE 512bit
 - 48-cores + 4 core OS
 - Tofu interconnect D



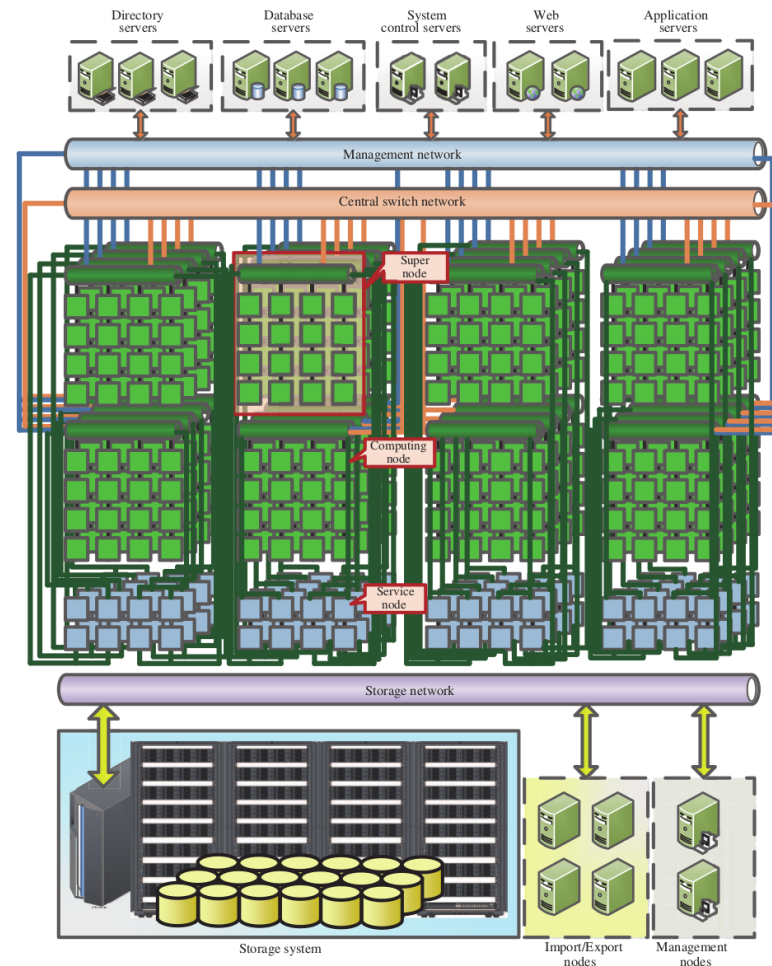
Top500 #4

- IBM Summit (at ORNL)
- Total: 95832 CPUs
 - 4356 nodes
 - 64bit RISC processors
 - 22-cores Power9 per chip
 - 6 volta Nvidia GPU/node



Top500 #6

- Sunway TaihuLight
- Total: 40,960 CPUs
 - SW26010
 - 64bit RISC processors
 - 256 cores per chip
 - 64KB scratchpad



Top500 Analysis



- First comments
 - 1 Exaflops was reached in June 2022
 - 10^{18} floating-point operations per second
 - Machines with lot of *cores*
 - Power consumption up to 29,9 Mwatts
 - Top 10 exhibits different system architectures
- Deeper analysis
 - Big difference between *Rmax* and *Rpeak*
 - Big difference between *Rmax* and *Power*
- Ordering based on power efficiency : **Green500**
 - Sort supercomputers according to the ratio power consumption / Linpack performance

Green500



R	Top500	System	Cores	Rmax	Power (kW)	Gflops/W
1	29	Frontier TDS	120,832	19.20	309	62.684
2	1	Frontier	8,730,112	1,102.00	21,100	52.227
3	3	LUMI	1,110,144	151.90	2,942	51.629
4	10	Adastra	319,072	46.10	921	50.028
5	326	MN-3	1,664	2.18	53	40.901



Green500

R	Top500	System	Cores	Rmax	Power (kW)	Gflops/W
6	315	SCC-21 scala...	16,704	2.27	103	33.983
7	319	Thetys	19,840	2.25	72	31.538
8	304	Wilkes-3	26,880	2.29	74	30.797
9	105	Athena	47,616	5.05	147	29.926
10	363	Phoenix – 2022	20,160	2.07	69	29.924



Green500 Analysis



- Main ordering
 - First machine is (not) the most *powerful*
 - Rank 29 in Top500, but it is a subpart of Rank 1 on Top500 !!!
 - 4 in top 10 of Top500 reach top 10 of Green500
 - 1st Top500 reaches 2nd Green500 !!!
 - 3rd Top500 reaches 3rd Green500
 - Specific architecture and GPU accelerators seem to be more efficient
- Top500 and Green500 limits
 - Linpack is a very specific benchmark
 - Regular computation (mainly linear algebra)
 - Few communications/synchronization between parallel units
- Need different benchmarks to classify supercomputers
 - Most powerful machines on irregular codes: Graph500
 - Based on graph traversal

Graph500 BFS (not up to date)

R	Top500	System	Nodes	Cores	Pb Scale	GTEPS
1		K	82,944	663,552	40	38,621.4
2		Sunway TaihuLight	40,768	105,9968 0	40	23,755.7
3		Sequoia	98,304	157,2864	41	23,751
4		Mira	49,152	786,432	40	14,982
5		JUQUEEN	16,384	262,144	38	5,848



Conclusion on Ranking



- Multiple ranking methods
 - Correspond to various needs
 - Highlight different architectures
- Where do the differences come from?
 - Various domains of applications
 - Depends on the target users
 - Impact on the design choices
 - Difference machine architectures
 - Processors, memory, network...

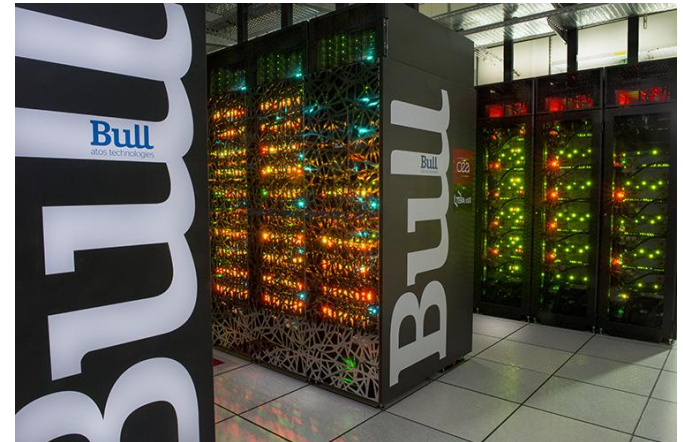
French Status

- Top500: 5th country w/ 22 systems
 - 4.4% of systems
 - 3.8 % of global performance

Rank	Site	System	Cores	Rmax	Rpeak	Power
10	Genci	Adastra	319,072	46.10	61.61	921
17	CEA	CEA-HF (Atos)	810,240	23.24	31.76	4,959
33	Genci	PANGEA III	291,024	17.86	25.03	1,367
34	CEA	Tera-1000-2 (Atos)	561,408	11.97	23.40	3,178
49	Meteo-France	Taranis (Atos)	294,912	8.19	10.32	1,621

French Ecosystem

- Main French Vendor:
 - Bull Atos
- Inside Top500
 - 5th vendors (number and performance)
 - 42 systems (8.4%)
 - 5.8% global performance
- Co-design with CEA



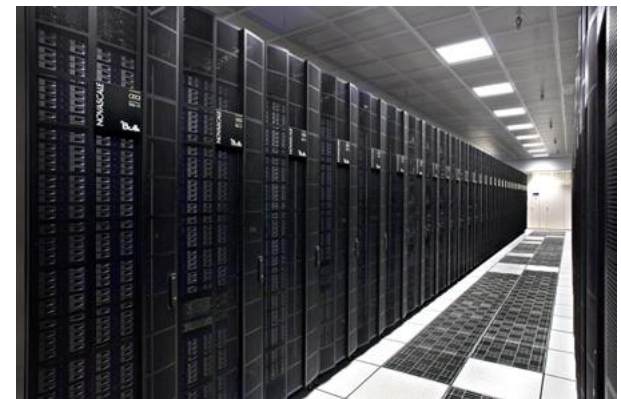
French Vision: Bull & CEA



- Co-design between Atos Bull & CEA
- Multiple machines inside Top500 made by BULL and hosted by CEA
- HPC at CEA
 - Mainly CEA/DAM (Bruyères-le-Châtel)
 - Different product lines

TERA

- Part of defense simulation program
- History
 - Program started in 1996
 - Predicted to set up 3 machines
- First machine: Tera 1 (HP/COMPAQ)
 - 2,560 cores (Alpha CPU, 1 GHz)
 - Quadrics interconnect
 - Linpack performance: 3.18 Tflop/s
 - Rank 4 in June 2002
- Second machine: Tera 10 (BULL)
 - 8,704 cores (Intel Itanium 2, 1.6GHz)
 - Quadrics interconnect
 - Linpack performance: 42.9 Tflop/s
 - Rank 5 in June 2006



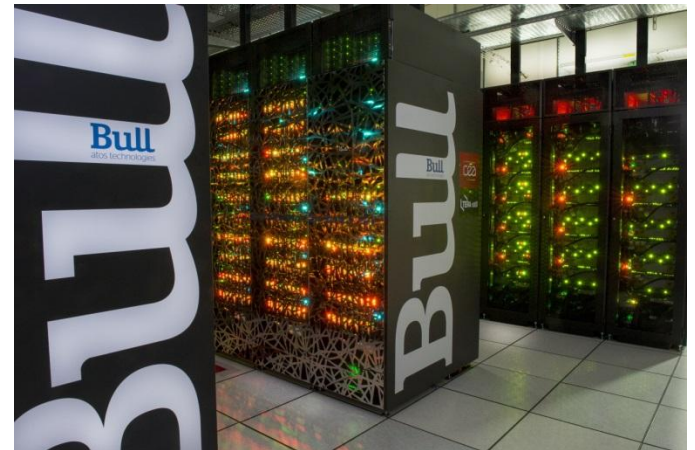
TERA Machine

- Tera 100 (Bull)
 - 140,000 cores (Intel Xeon Nehalem)
 - 4,300 compute nodes
 - IB QDR interconnect
 - Linpack performance: 1,050 TFlop/s
 - Rank 6 in November 2010



TERA Machine

- Tera 1000-1
 - 70,272 cores (Intel Xeon Haswell)
 - 4,39 2compute nodes
 - IB FDR interconnect
 - Linpack performance: 1,871 TFlop/s
 - Rank 44 in June 2016
- Tera 1000-2
 - 561,408 cores (Intel Xeon Phi KNL)
 - 8,256 compute nodes
 - BXI-1.2 interconnect
 - Linpack performance: 11,965.5 TFlop/s
 - Rank 14 in June 2018
 - Rank 20 in June 2020



Exa Machine

- Exa1-HF (Atos)
 - 810,240 cores (AMD EPYC)
 - Atos BXI v2
 - Linpack performance: 23.24 PFlop/s
 - 84th in Graph 500
 - Rank 14 in November 2021
 - Now Rank 17
- Next step
 - ...



CCRT



- Research and Technology Computing Center
 - *Centre de calcul pour la recherche et la technologie*
- French consortium
 - Started in 2003
 - Based on french academic & industry
- Goals
 - Provide High Performance Computing resources for large scientific computations
 - Foster a real synergy between research organizations, universities and industry
 - Promote exchanges and scientific collaboration between partners.



PRACE

- Partnership for Advanced Computing in Europe
 - European Consortium
- 25 member countries
- 5 PRACE centers
 - BSC (Spain)
 - CINECA (Italy)
 - CSCS (Switzerland)
 - GCS (Germany)
 - GENCI (France)
- Currently
 - French machine Joliot-Curie
 - Located in TGCC (Bruyères-le-Chatel)





Current GENCI machine



- Joliot-Curie Rome (Atos Bull)
 - Total: 197,120 compute cores (AMD ROME)
 - Node w/ dual-socket (28 cores per node)
 - IB Mellanox HDR interconnect
 - Rank 33 in June 2020
 - 6,988 Pflops
- Joliot-Curie SKL (Atos Bull)
 - Total: 79,488 compute cores (Xeon Skylake)
 - IB Mellanox EDR interconnect
 - Rank 61 in June 2020
 - 4,065.6 Pflops